Schleifen – Grundlagen

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- switch-Anweisung
- switch-Ausdruck
- Unterschiede zwischen if-else und switch

Was wird in diesem Foliensatz besprochen?

- Schleifen allgemein
- while-Schleife in Java
- do-while-Schleife in Java
- for-Schleife in Java

Was lernen Sie kennen?

- Schleifen in Java
- Unterschiede zwischen Schleifen in Java

Motivation

Bisher fast nur Anweisungsfolgen (mit Verzweigungen)

Viele Anweisungsfolgen müssen aber wiederholt ausgeführt werden

Wiederholung muss im Programm formuliert werden

Schleifenvarianten in Java

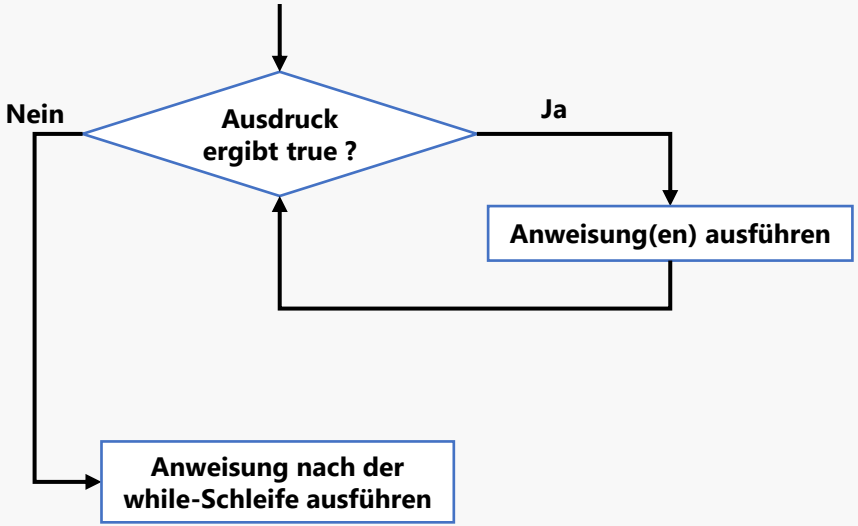
while	do-while	for

while (1)

- Allgemeine Form

```
while (Ausdruck)
    Anweisung(en)
```

- Ausführung



while (2)

Beliebiger Teil des Ausdrucks sollte in Anweisung(en) verändert werden

- Ansonsten kann der Wert des Ausdrucks im Schleifenkopf z.B. nicht von wahr auf falsch wechseln (Endlosschleife)

Weitere Bezeichnungen

- Abweisschleife
- Vorprüfende oder kopfgesteuerte Schleife

Beispiel (while)

```java
public class WhileTest {

    public static void main(String[] args) {
        int i = 0;
        System.out.println("Start:");
        while (i < 10) {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
            i++;
        }
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Beispiel (unbestimmte Anzahl an Schleifendurchläufen)

```
int n = 17;
int count = 0;
while (n >= 1) {
    n /= 2;
    count++;
}
System.out.println(count);
```

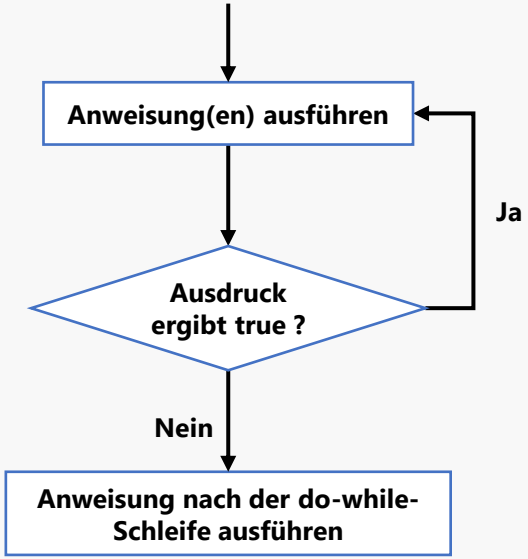
- Zählen, wie oft eine Zahl größer 0 durch 2 dividiert werden kann (Hinweis: zählt die Anzahl der Binärstellen einer Zahl größer 0)
- Input (in diesem Fall n) bestimmt die Anzahl der Schleifendurchläufe

do-while (1)

- Allgemeine Form

```
do
    Anweisung(en)
while (Ausdruck);
```

- Ausführung



do-while (2)

Hinweis
• Anweisung wird **mindestens einmal** ausgeführt

Weitere Bezeichnungen
• Durchlaufschleife • Nachprüfende oder fußgesteuerte Schleife

Beispiel (do-while)

```java
public class DoWhileTest {

    public static void main(String[] args) {
        int i = 0;
        System.out.println("Start:");
        do {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
            i++;
        } while (i < 10);
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Unterschied zwischen while und do-while

- Beispiel: Die Ziffern einer nicht negativen Zahl i in umgekehrter Reihenfolge ausgeben

Lösung mit while-Schleife

```
…
while (i > 0) {
    System.out.println(i % 10);
    i = i / 10;
}
…
```

Lösung mit do-while-Schleife

```
…
do {
    System.out.println(i % 10);
    i = i / 10;
} while (i > 0);
…
```

Lösung mit `while`-Schleife hat ein Problem (0!)
Hinweis: i >= 0 löst hier nicht das Problem!

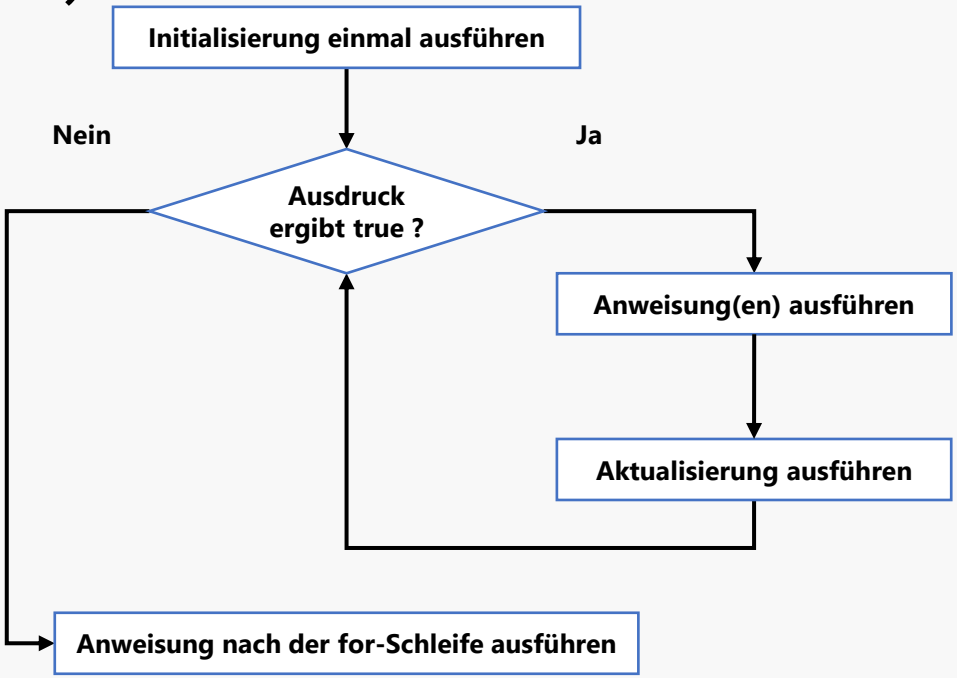
Lösung mit `do-while`-Schleife ist korrekt

for (1)

- Allgemeine Form

```
for (Initialisierung; Ausdruck; Aktualisierung)
    Anweisung(en)
```

- Ausführung



for (2)

Jeder der drei Teile im Schleifenkopf kann weggelassen werden

- https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html#jls-14.14.1

Weitere Bezeichnung

- Zählschleife (Sonderform der vorprüfenden Schleife)

Beispiel (for)

```java
public class ForTest {

    public static void main(String[] args) {
        System.out.println("Start:");
        for (int i = 0; i < 10; i++) {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
        }
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Sichtbarkeit von Variablen (Ausblick)

Sichtbarkeitsbereich einer Variable
• Programmstück, in dem auf die Variable zugegriffen werden kann

Variable in einem Block (z. B. Block einer for-Schleife)
• Sichtbar von der Deklaration bis zum Ende des Blocks, in dem sie deklariert wurde

Sichtbarkeit von Variablen (Beispiel)

```
for (int i = 0; i < 100; i++) {
    …
    int x = 20;
    …
}
…
System.out.println(i + x);
```

Ab hier ist i in der Schleife bekannt

Ab hier ist x im aktuellen Schleifendurchlauf bekannt

Hier sind i und x **nicht mehr** bekannt

Gibt einen Übersetzungsfehler!

Beispiele (Zählschleifen)

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

Zahlen von 0 bis 9 auf der Konsole ausgeben

```
for (int i = 10; i >= 0; i--) {
    System.out.println(i);
}
```

Zahlen von 10 bis 0 auf der Konsole ausgeben

```
for (long i = 1; i < 1000000000000L; i *= 3) {
    System.out.println(i);
}
```

Alle Potenzen von 3, die kleiner als 10^{12} sind, ausgeben

Ein Beispiel – 10! berechnen

`int factorial = 1;`
`for (int i = 2; i <= 10; i++) {`
`  factorial = factorial * i;`
`}`
	• Einfache, lesbare Variante
	• Typische for-Schleife

`int factorial, i;`
`for (factorial = 1, i = 2; i <= 10; factorial = factorial * i, i++)`
`  ;`
	• **Mehrere, durch Beistrich getrennte, Ausdrücke in der Initialisierung und Aktualisierung**
	• **Kein Schleifenrumpf notwendig**
	• Schlechter lesbar

`for (int factorial = 1, i = 2; i <= 10; factorial = factorial * i++);`	• Möglich, aber **sinnlos** • Variable `factorial` ist außerhalb der Schleife nicht verwendbar

`int factorial = 1, i = 2;`
`for (; i <= 10;) {`
`  factorial = factorial * i++;`
`}`
	• Initialisierung und Aktualisierung bleiben leer
	• Initialisieren bei der Deklaration
	• **Aktualisieren im Schleifenrumpf**

Beispiel – Konvertierung in Binärzahl (1)

Konvertiere ganze positive Zahl n in die Binärdarstellung

Beispiel

- n = 105 -> Ergebnis: 1101001

Ablauf

- Immer wieder durch 2 dividieren (bis Ergebnis = 0)
  - Rest (0 oder 1) berücksichtigen
- Ergebnis der Division im nächsten Schritt benutzen
- Aber: Ausgabe im Ablauf ergibt verkehrtes Muster

Operation	Rest
105 : 2 = 52	1
52 : 2 = 26	0
26 : 2 = 13	0
13 : 2 = 6	1
6 : 2 = 3	0
3 : 2 = 1	1
1 : 2 = 0	1

Beispiel – Konvertierung in Binärzahl (2)

```java
public class BinarySimple {

    public static void main(String[] args) {
        int n = 105;
        while (n > 0) {
            if (n % 2 == 0) {
                System.out.print("0");
            } else {
                System.out.print("1");
            }
            n /= 2;
        }
    }

}
```

```
1001011
```

Einfache Lösung mit **verkehrter** Ausgabe

Beispiel – Konvertierung in Binärzahl (3)

```java
public class Binary {
    public static void main(String[] args) {
        int n = 105;
        int power = 1;
        while (power <= n / 2) {
            power *= 2;
        }
        while (power > 0) {
            if (n < power) {
                System.out.print(0);
            } else {
                System.out.print(1);
                n -= power;
            }
            power /= 2;
        }
    }
}
```

power auf die größte 2er-Potenz <= n setzen

Überprüfen, ob eine bestimmte 2er-Potenz (von groß zu klein) in n vorhanden ist. Wenn vorhanden, dann wird 1 ausgegeben und diese 2er-Potenz subtrahiert, ansonsten wird 0 ausgegeben.

```
1101001
```

Schleifenrumpf bei Schleifen

Schleifenrumpf immer als Block implementieren

- Klammern mit Einrückungen erhöhen Lesbarkeit
- Schleifenrumpf kann leicht erweitert werden

break und continue

break	continue
• Führt zum sofortigen Ausstieg aus dem aktuellen Schleifendurchlauf und damit zum Ausstieg aus der Schleife	• Überspringt die restlichen Anweisungen im aktuellen Schleifendurchlauf und geht wieder zum Ausdruck

break und continue – Vergleich (for-Schleife)

```java
int n = 10;
for (int i = 1; i < n; i++) {
    if (i % 5 == 0) {
        break;
    }
    System.out.println(i);
}
```

Wenn i den Wert 5 hat, wird **die Schleife** abgebrochen

```
1
2
3
4
```

```java
int n = 10;
for (int i = 1; i < n; i++) {
    if (i % 5 == 0) {
        continue;
    }
    System.out.println(i);
}
```

Wenn i den Wert 5 hat, wird **der Schleifendurchlauf** abgebrochen. Wichtig: Das Update i++ wird aber ausgeführt!

```
1
2
3
4
6
7
8
9
```

break und continue – Vergleich (while-Schleife)

```java
int n = 10;
int i = 1;
while (i < n) {
    if (i % 5 == 0) {
        break;
    }
    System.out.println(i);
    i++;
}
```

Wenn i den Wert 5 hat, wird **die Schleife** abgebrochen

```
1
2
3
4
```

```java
int n = 10;
int i = 1;
while (i < n) {
    if (i % 5 == 0) {
        i++;
        continue;
    }
    System.out.println(i);
    i++;
}
```

Wenn i den Wert 5 hat, wird **der Schleifendurchlauf** abgebrochen. Wichtig: Das Update i++ muss explizit davor ausgeführt werden!

```
1
2
3
4
6
7
8
9
```

Beispiel (Primzahltest mit vorzeitigem Abbruch)

```java
public class PrimeTest {

    public static void main(String[] args) {
        long n = 97; // Simulate input data
        boolean isPrime = n >= 2;
        for (long factor = 2; factor < n; factor++) {
            if (n % factor == 0) {
                isPrime = false;
                break;
            }
        }
        if (isPrime) {
            System.out.println(n + " is prime");
        } else {
            System.out.println(n + " is not prime");
        }
    }

}
```

An dieser Stelle ist das Ergebnis klar und die Schleife muss nicht weiter ausgeführt werden

break und continue

- In Schleifen **nur sparsam** verwenden
- Können übermäßige Verschachtelungen vermeiden
- Können oft durch eine alternative Formulierung des Ausdrucks umgangen werden

Beispiel für break und seine Vermeidung (1)

```
int n = 1000;
int factor = 2;
while (factor <= n) {
    if (n % factor == 0) {
        break;
    }
    factor++;
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

Abbruch, wenn ein Faktor != 1 gefunden wurde.
Mit break kürzer als untere Version.

```
int n = 1000;
int factor = 2;
boolean found = false;
while (factor <= n && !found) {
    if (n % factor == 0) {
        found = true;
    } else {
        factor++;
    }
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

Version ohne break: Benötigt eine zusätzliche Variable und ist textuell länger. Dafür kann der Ausstieg aus der Schleife nur an einer Stelle geschehen (Überprüfung der Schleifenbedingung).

Beispiel für break und seine Vermeidung (2)

```
int n = 1000;
int factor = 2;
while (n % factor != 0) {
    factor++;
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

Kein break notwendig

Nicht immer so einfach möglich!